

RELATÓRIO DE MANUTENIBILIDADE DO PROJETO OBSERVAÇÃO

1. Introdução

O projeto Observacao-frontend possui aproximadamente 4.300 linhas de código analisadas na branch main, distribuídas entre os diretórios frontend/, backend/server/ e o arquivo start.sh. A análise do SonarQube Cloud revela um projeto funcional, mas com dívida técnica acumulada em múltiplas dimensões: segurança, confiabilidade, duplicação e complexidade. O ponto mais crítico e estratégico é a ausência total de cobertura de testes automatizados, que compromete qualquer esforço de melhoria sem uma rede de segurança para refatorações. No geral, o projeto está em um estado que demanda atenção antes que a dívida técnica se torne inviável de pagar.

2. Segurança — Nota B | 4 vulnerabilidades abertas

Foram identificadas 4 vulnerabilidades do tipo "Make sure not using resource integrity feature is safe here", todas de severidade Minor/Low, localizadas nos arquivos frontend/dist/index.html e frontend/index.html (linhas L9 e L10 em ambos). A nota B indica que há risco real identificado, embora não crítico.

O problema ocorre pela ausência do atributo integrity (Subresource Integrity — SRI) em tags que carregam recursos externos, como scripts de CDN. Isso significa que, se um CDN utilizado for comprometido, código malicioso pode ser injetado e executado no navegador dos usuários sem qualquer validação. A gravidade prática depende de quais recursos externos estão sendo carregados. O fato de o arquivo dist/index.html ser um artefato de build indica que a origem do problema está na configuração do bundler (Vite ou Webpack), não no arquivo em si — corrigir manualmente o HTML gerado não resolve a causa raiz.

Para melhorar, o caminho correto é configurar o bundler para gerar automaticamente os atributos SRI com hash SHA-384 em todos os recursos externos. O SonarQube estima 5 minutos de esforço por issue, mas a correção sistêmica via bundler pode levar algumas horas e elimina o problema permanentemente.

Além disso, há 1 Security Hotspot com 0% de revisão, o que resulta no Security Review Rating E — a pior nota possível. Importante notar que hotspots não são vulnerabilidades confirmadas: são pontos que exigem revisão manual. A nota E existe simplesmente porque o item nunca foi revisado. Basta acessar a aba de Security Hotspots, avaliar o contexto e marcá-lo como "Safe" (se não representar risco real) ou abrir um issue

para correção. Este é um problema de processo, não necessariamente de código, e pode ser resolvido em menos de duas horas.

3. Confiabilidade — Nota B | 60 issues, 6 bugs, 3 de severidade High

A confiabilidade é a dimensão com maior número de problemas de impacto real. Os 60 issues incluem 6 bugs confirmados e 3 ocorrências de alta severidade, todas no arquivo `start.sh`: o uso de `[ ]` (single bracket) ao invés de `[ [ ] ]` (double bracket) em testes condicionais Bash. Essa distinção importa: o `[ ]` não lida bem com variáveis não definidas ou com espaços nos valores, podendo causar falhas silenciosas ou comportamento inesperado na inicialização da aplicação. O script `start.sh` é um ponto crítico de infraestrutura — uma falha aqui pode impedir que a aplicação suba em produção.

A causa provável é que o script foi escrito por alguém não familiarizado com boas práticas de shell scripting moderno. A correção é trivial (trocar `[ ]` por `[ [ ] ]` nas linhas L37 e L49) e o esforço estimado é de 2 minutos por ocorrência. Apesar disso, o impacto de não corrigir pode ser alto dependendo do ambiente de deploy.

O restante dos issues de confiabilidade está distribuído no frontend/ e representa risco de comportamento incorreto em runtime, especialmente em condições de borda não cobertas por testes. Sem cobertura de testes, é impossível quantificar o real impacto desses bugs.

4. Manutenibilidade — Nota A (com ressalvas importantes) | 69 issues

A nota A em Manutenibilidade pode dar uma falsa sensação de segurança. O SonarQube a calcula com base no ratio de dívida técnica pelo tamanho do projeto. Com 4.300 linhas, o limiar é flexível. Mas 69 code smells abertos, sendo 3 de severidade High e 1 Critical, contam uma história diferente.

Os casos mais relevantes são: a função em `useFormValidation.ts` (L58) com Complexidade Cognitiva de 20, acima do limite permitido de 15 (10 minutos de esforço estimado); a função em `Dashboard.tsx` (L43) com Complexidade Cognitiva de 19, também acima do limite (9 minutos estimados); e um tipo duplicado em `api.ts` (L103) que deveria ser centralizado ou removido (1 minuto de esforço). Esses três arquivos sozinhos são responsáveis pelos issues mais graves da dimensão.

A causa raiz é crescimento orgânico do código sem refatoração periódica. Funções que nasceram simples foram sendo incrementalmente expandidas com novos casos, condicionais e tratamentos especiais, sem que ninguém se responsabilizasse por decompô-las. Isso é muito comum em projetos que crescem sob pressão de entrega. O impacto direto é na produtividade. Funções com alta complexidade cognitiva demoram mais para ser compreendidas, revisadas e corrigidas por qualquer membro do time.

5. Duplicação e Complexidade — os sinais mais preocupantes a longo prazo

A taxa de duplicação de 10,7% (522 linhas, 18 blocos, 2 arquivos duplicados) está totalmente concentrada no frontend/, que individualmente apresenta 11,5% de duplicação. O backend/server e start.sh têm 0%. O limiar considerado crítico pela maioria dos times é 5% — este projeto está mais que o dobro. A causa é: copy-paste de código ao invés de criação de abstrações reutilizáveis (hooks, componentes genéricos, utilitários). Isso gera uma dívida específica: quando um bug existe em código duplicado, ele precisa ser corrigido em múltiplos lugares, e frequentemente um dos lugares é esquecido, gerando inconsistências.

A complexidade é igualmente concentrada. Toda a complexidade ciclomática (553) e cognitiva (232) do projeto vive no frontend/. O backend/server marca zero. Isso sugere que o frontend cresceu sem arquitetura clara, com lógica de negócio embutida nos componentes ao invés de separada em camadas (hooks, services, utils). Para contextualizar: uma complexidade ciclomática de 553 significa que, para testar completamente todas as funções do frontend, seriam necessários pelo menos 553 casos de teste diferentes, e o projeto não tem cobertura configurada.

A melhoria passa por refatoração ativa: extrair lógica complexa para hooks e funções utilitárias, aplicar early return para reduzir aninhamento de condicionais, e adotar o princípio de responsabilidade única. Uma sprint dedicada a qualidade técnica, com foco nos arquivos de maior complexidade e nos blocos duplicados mais volumosos, pode reduzir significativamente esses números.

6. Cobertura de Testes — o risco estratégico central

A cobertura de testes não está configurada. Não há visibilidade sobre a existência ou qualidade de testes automatizados neste projeto. Este é, isoladamente, o problema de maior impacto estratégico identificado na análise.

Sem cobertura, todas as outras melhorias descritas neste relatório ficam comprometidas: refatorar uma função com complexidade 20 sem testes é apostar que nenhum dos caminhos de execução foi alterado inadvertidamente. Corrigir os 6 bugs de confiabilidade sem testes é não ter como verificar se a correção funciona em todos os cenários. A ausência de testes não é apenas um problema de qualidade, é um multiplicador de risco em todas as outras dimensões.

O caminho de recuperação começa pela configuração da análise de cobertura no pipeline de CI/CD (integração de Jest com Icov para TypeScript/React é o padrão do ecossistema). A partir daí, a meta inicial deve ser 60% de cobertura, priorizando as funções de maior complexidade, justamente useFormValidation.ts e Dashboard.tsx, que já foram identificadas como problemáticas. A criação de testes para essas duas funções serve tanto para aumentar a cobertura quanto para forçar a refatoração, já que funções complexas são naturalmente difíceis de testar.

7. Conclusão e Prioridades

O projeto Observacao-frontend está em um ponto em que ainda é possível pagar a dívida técnica acumulada com um esforço focado, mas se o crescimento continuar no ritmo atual sem intervenção, a manutenção ficará progressivamente mais cara e arriscada.

As prioridades imediatas são: (1) revisar o Security Hotspot para sair do Rating E; (2) corrigir o start.sh, pois é um risco de infraestrutura com correção trivial; (3) configurar a cobertura de testes no pipeline. Na sequência, uma sprint dedicada de qualidade técnica deve endereçar a duplicação no frontend/ e a complexidade das funções críticas, configurando também um quality gate que bloqueie novos PRs com duplicação acima de 3% ou complexidade cognitiva por função acima de 15. Essas medidas preventivas são tão importantes quanto as corretivas, e sem elas, o padrão atual tende a se repetir.

The screenshot displays the SonarQube cloud interface for the 'Observacao-frontend' project. The left sidebar contains navigation links for Overview, Analysis, Summary (selected), Issues, Architecture, Security hotspots, Reporting, Measures, Activity, Policies, Intended architecture, Quality profiles, Quality gate, Project, Pull Requests, and Branches. The main content area shows the 'Summary' page for the 'main' branch. It includes a 'New Code' and 'Overall Code' tab. The 'Overall Code' tab displays several metrics: Security (4 Open issues, Rating B), Reliability (60 Open issues, Rating B), Maintainability (69 Open issues, Rating A), Accepted Issues (0), Coverage (10.7%, with a note that a few extra steps are needed for SonarQube Cloud to analyze code coverage), and Duplications (10.7%, with a note that no conditions are set on 4.9k Lines). A 'Security Hotspots' section shows 1 hotspot.

The screenshot displays the SonarQube cloud interface for the 'Observacao-frontend' project, specifically the 'Issues' page. The left sidebar is the same as the previous screenshot. The main content area shows the 'Issues' page for the 'main' branch. It includes a 'Filters' section on the left with 'Clear All Filters' and a list of filters: Software quality (1), Security (0), Reliability (3), and Maintainability (0). Under 'Severity', there are 'Blocker' (0), 'High' (3), 'Medium' (0), 'Low' (0), and 'Info' (0). The main list of issues shows three items, all with a 'High' severity and 'Reliability' category. Each issue has a description, a 'Consistency' button, and a 'Reliability' button. The first issue is 'Use '[' instead of '[' for conditional tests. The '[' construct is safer and more feature-rich.' (L37, 2min effort, 3 days ago, Code Smell, Major). The second and third issues are identical to the first.

Observacao-frontend

Project

Overview

Analysis

Summary

Issues

Architecture

Security hotspots

Reporting

Measures

Activity

Policies

Intended architecture

Quality profiles

Quality gate

Project

Pull Requests

Branches

Favorite projects

Assigned issues

Explore

SonarQube

cloud

LucasGddoy / Observacao-frontend / Measures

main

Measures

New Code

New Security Hotspots 0

Rating A

Hotspots Reviewed 100%

Overall Code

Security Hotspots 1

Rating E

Hotspots Reviewed 0.0%

Coverage

Duplications

Observacao-frontend

View as Tree

3 files

Security Review Rating E See history

New code: since previous version

backend/server

frontend

start.sh

3 of 3 shown

© 2018-2026 SonarSource Sàrl. All rights reserved.

Terms Pricing Privacy Cookie Policy Security Community Documentation

Contact us Status About

Observacao-frontend

Project

Overview

Analysis

Summary

Issues

Architecture

Security hotspots

Reporting

Measures

Activity

Policies

Intended architecture

Quality profiles

Quality gate

Project

Pull Requests

Branches

Favorite projects

Assigned issues

Explore

SonarQube

cloud

LucasGddoy / Observacao-frontend / Measures

main

Measures

Duplications

Overview

New Code

Duplicated Lines 0

Duplicated Blocks 0

Overall Code

Density 10.7%

Duplicated Lines 522

Duplicated Blocks 18

Duplicated Files 2

Observacao-frontend

View as Tree

3 files

Duplicated Lines 522 See history

New code: since previous version

backend/server

frontend

start.sh

Duplicated Lines 0 0.0%

522 11.5%

0 0.0%

3 of 3 shown

© 2018-2026 SonarSource Sàrl. All rights reserved.

Terms Pricing Privacy Cookie Policy Security Community Documentation

Contact us Status About

Observacao-frontend

Project

Overview

Analysis

Summary

Issues

Architecture

Security hotspots

Reporting

Measures

Activity

Policies

Intended architecture

Quality profiles

Quality gate

Project

Pull Requests

Branches

Favorite projects

Assigned issues

Explore

SonarQube

cloud

LucasGddoy / Observacao-frontend / Issues

main

Issues

Filters

Clear All Filters

Software quality 1

Security 0

Reliability 0

Maintainability 3

Add to selection Ctrl + click

Severity 1

Blocker 0

High 3

Medium 0

Low 0

Info 0

Add to selection Ctrl + click

frontend/src/hooks/useFormValidation.ts

Refactor this function to reduce its Cognitive Complexity from 20 to the 15 allowed.

Maintainability High

Adaptability brain-overload

Open Not assigned L58 10min effort 2 days ago Code Smell Critical

frontend/src/pages/Dashboard.tsx

Refactor this function to reduce its Cognitive Complexity from 19 to the 15 allowed.

Maintainability High

Adaptability brain-overload

Open Not assigned L43 9min effort 12 hours ago Code Smell Critical

frontend/src/services/api.ts

Remove this duplicated type or replace with another one.

Maintainability High

Intentionality No tags

Open Not assigned L103 1min effort 1 day ago Code Smell Critical